

day09笔记

一、JavaScript语言

1、JavaScript 简称 js，用来做网页交互效果

2、交互效果

- 下拉菜单
- 轮播图
- 购物车
- 商品列表渲染
- 登录注册验证
- 前后端交互

3、JavaScript既可以做前端里面的交互效果，也可以写后端（node.js）

二、Node.js

1、Node 是代码执行环境（后端环境）

2、openclaw（小龙虾）安装，依赖 node.js

3、opencode（国内版本）安装，依赖 node.js

4、很多相关的AI工具环境配置，都需要依赖 node.js

5、node.js 开发使用的语言就是JavaScript

三、JavaScript运行

1、python语言代码执行，首先得安装一个python解析器

2、JavaScript语言代码执行，不需要你额外安装任何其他的环境，只要有浏览器就可以

3、JavaScript是内嵌到浏览器里面的，所以你不需要额外安装其他的环境。浏览器只能解析三种语言，分别是html、css、js

4、编写JavaScript，只需要在html文件中使用 `<script>` 标记就可以

5、js输出方法

```
1 <script>
2 // js 输出方法
3 // document.write() 在网页中输出文本信息，可以解析标记
4 // document.write('hello, 美女帅哥! 1<br>')
5 // document.write('hello, 美女帅哥! 2<br>')
6 // document.write('hello, 美女帅哥! 3')
7
8 // document.write('<h1>老大哥来了</h1>')
9 // document.write('<p>小老弟来了</p>')
10
```

```

11 // alert() 页面弹窗，不支持标记解析
12 // alert('温馨提示：此网站可能存在风险... \n 丁致宇就是喜欢访问')
13
14 // console.log() 可以在浏览器控制台输出信息
15 // console.log('hello, 我只能在控制台里面看到')
16 // 错误信息
17 // console.error('第18行代码存在语法错误')
18 // 警告信息
19 // console.warn('hello')
20
21 // 打印方法
22 // print('打印文件')
23 // document.querySelector('button').onclick = ()=>{
24 //     print()
25 // }
26 </script>

```

四、JavaScript注释

1、单行注释

语法

```

1 // 代码意思

```

2、多行注释

语法

```

1 /*
2     代码意思
3 */

```

五、变量

1、声明变量

```

1 // 1、使用 var 关键字来声明
2 // console.log(a) // undefined, 告诉咱们没有赋值，不是这个变量未定义
3 // var a = 10
4 // console.log(a)
5
6 /*
7 问题：变量声明提升
8 var a --- 变量声明
9 console.log(a) --- 变量已经声明了，只是没有值，所以返回 undefined
10 a = 10 --- 变量赋值
11 console.log(a) --- 10，因为已经赋值了
12 */
13 // var c // 只是声明没有赋值，在js里面之前是运行存在的

```

```

14 // console.log(c) // undefined
15
16 // 允许重复声明
17 // var a = 10
18 // var a = 20
19 // console.log(a)
20
21
22 // 2、使用 let 关键字来声明，不允许变量声明提升，不能再前面输出
23 // 不允许重复声明
24 // console.log(b) // 报错，这种设计才是合理的
25 // let b = 20
26 // console.log(b)
27
28 // let b
29 // console.log(b) // undefined
30
31 // 3、使用 const 关键字来声明，声明常量（就是不能被修改的数据）
32 // let username = '张三'
33 // username = '王五'
34 // console.log(username)
35
36 const username = '张三'
37 // 不能被修改，因为是常量
38 username = '王五'
39 console.log(username)

```

2、数据类型

- 基本（原始）数据类型
- number 数子类型
- string 字符串类型
- boolean 布尔类型（true、false）
- undefined 未定义类型
- null 空类型

引用（复合、复杂）数据类型

- array 数组类型
- object 对象类型
- function 函数类型

查看数据类型

- typeof()

六、js基本数据类型

```

1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">

```

```

5     <meta name="viewport" content="width=device-width, initial-scale=1.0">
6     <title>Document</title>
7 </head>
8 <body>
9     <script>
10         // number
11         // let data = 10
12         // let data = 12.66
13         // console.log(data)
14         // console.log(typeof(data), typeof data)
15         // console.log(typeof 10) // number
16
17         // string
18         // let data = 'abc'
19         // console.log(data)
20         // console.log(typeof data) // string
21
22         // boolean
23         // let data = true
24         // console.log(data)
25         // console.log(typeof data) // boolean
26
27         // console.log(10 > 20) // false
28         // console.log(10 < 20) // true
29
30         // undefined 未定义，当一个变量声明了，没有赋值的时候返回 undefined
31         // 在js里面，变量声明其实内部在构建的时候分为两步，1、变量声明 2、变量赋值
32         // let data
33         // console.log(data) // undefined
34         // console.log(typeof data) // undefined
35
36         // null 空类型，起了一个占位的作用，不知道当前这个变量值的类型是什么的时候，可以使用null先
        占个位，作为初始值
37         let data = null
38         console.log(data)
39         console.log(typeof data) // object
40     </script>
41 </body>
42 </html>

```

七、数组类型

1、array 数组，是一个可以存储多个数据的容器

```
1 | let arr = ['a', 'b', 'c']
```

2、索引（下标、index）

数组在创建的时候会默认为每一个元素添加一个编号，用于数组的操作，默认是从0开始的，把这个编号称之为索引

```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>Document</title>
7  </head>
8  <body>
9      <script>
10         // 创建数组
11         // let arr = ['a', 'b', 'c']
12         // console.log(arr)
13         // console.log(typeof arr) // object
14
15         let arr = ['a', 'b', 'c', '666', '呵呵']
16         console.log(arr)
17         console.log(arr[0])
18         console.log(arr[1])
19         console.log(arr[2])
20         console.log(arr.length) // 3 表示的是数组的长度
21         console.log(arr[arr.length-1]) // 表示动态的拿到数组的最后一个元素
22     </script>
23 </body>
24 </html>

```

3、数组操作

```

1  let arr = ['a', 'b', 'c', '666', '呵呵']
2  // 增加操作
3  arr[5] = 'abc'
4  // 修改操作
5  arr[0] = '张三'
6  // 删除操作
7  delete arr[1]
8  console.log(arr)

```

4、数组方法

```

1  // 数组方法
2  let arr = ['a', 'b', 'c', '666', '呵呵']
3  // 数组添加方法（尾部）
4  // arr.push('嘿嘿')
5  // 数组添加方法（头部）
6  // arr.unshift('呵')
7
8  // 删除方法（尾部）
9  // arr.pop()
10 // 删除方法（头部）
11 // arr.shift()
12
13 /*

```

```

14 splice() 组合方法
15 + 参数1表示从什么位置开始
16 + 参数2表示删除的项数
17 + 参数3表示添加
18 */
19 // 删除操作
20 // arr.splice(0, 1)
21 // arr.splice(0, 2)
22
23 // 替换操作
24 // arr.splice(0, 1, '张三')
25
26 // 添加操作（插入）
27 // 注意点：当进行插入元素操作的时候，第二个参数必须设置为0，不然就变成替换操作了
28 // arr.splice(1, 0, '张三', '王五')
29
30 console.log(arr)
31 // slice() 截取方法，不包含结束位置
32 result = arr.slice(0, 3)
33
34 console.log(result)

```

八、函数类型

1、语法

```

1 // 函数声明
2 function fn(){
3     console.log('hello, 我是函数')
4 }
5
6 // 函数调用
7 fn()

```

2、函数参数

```

1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4     <meta charset="UTF-8">
5     <meta name="viewport" content="width=device-width, initial-scale=1.0">
6     <title>Document</title>
7 </head>
8 <body>
9     <script>
10         // function add(a, b){
11             //     console.log(a + b)
12         // }
13         // add(10, 20)
14
15         // 函数参数的默认值

```

```

16         function login(username='admin', password){
17             document.write('你的用户名:' + username)
18             console.log(password)
19         }
20         login('张三')
21     </script>
22 </body>
23 </html>

```

3、函数返回值

```

1  // 函数返回值
2  function add(a, b){
3      return a + b
4  }
5  let result = add(10, 20)
6  console.log(result)
7
8  // 注意点：如果一个函数没有返回值的话，接收的是undefined
9  function fn(){
10     console.log('hello')
11 }
12 console.log(fn()) // undefined

```

九、计算器

```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>Document</title>
7  </head>
8  <body>
9      <input type="text" class="txt1">
10     +
11     <input type="text" class="txt2">
12     <button class="calc">=</button>
13     <input type="text" class="result">
14
15
16     <button class="clear">clear</button>
17     <script>
18         // 第一步：要操作标记做某些事情，必须先把标记拿到
19         // document.querySelector() 根据css选择器拿到标记
20         // 注意点：如果没有拿到对应的标记，那么返回的是null
21         // console.log(document.querySelector('.txt1'))
22         // console.log(document.querySelector('.txt2'))
23         // console.log(document.querySelector('button'))
24

```

```

25     const txt1 = document.querySelector('.txt1')
26     const txt2 = document.querySelector('.txt2')
27     const btn = document.querySelector('.calc')
28     const result = document.querySelector('.result')
29     const clear = document.querySelector('.clear')
30
31     // 第二步：需要把表单框里面的数据拿出来进行计算
32     // input表单框里面的数据是存储在value属性里面的
33     // 语法：获取到的标记.value
34     // console.log(txt1.value)
35
36     // 第三步：要进行计算需要拿到的value值应该是输入后的
37     // 通过事件调用函数方式
38     // 事件指的是用户在页面中一些列的操作，例如：鼠标点击、双击、移入移出...
39     // 函数特点：不调用不会执行
40     // 表示当用户点击按钮这个标记的时候，事件才会去调用函数，实现了用户输入后再去获取value的操作
41
42     btn.onclick = function(){
43         // 注意点：input里面输入的数据，类型是string
44         // Number() 可以把其他类型转成数字类型
45         result.value = Number(txt1.value) + Number(txt2.value)
46     }
47
48     // 当点击按钮的时候清空input的输入的值
49     clear.onclick = function(){
50         txt1.value = ''
51         txt2.value = ''
52         result.value = ''
53     }
54 </script>
55 </body>
56 </html>

```

十、选择和循环结构

1、选择结构

```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>Document</title>
7  </head>
8  <body>
9      <script>
10         // 单分支结构
11         // let a = 10
12         // let b = 20
13         // if(a > b){
14             console.log('执行这里的代码')

```

```

15         // }
16
17     // 双分支结构
18     // prompt() 可以在网页中输入内容，通过变量来接收
19     // let a = prompt('请输入一个数字:')
20     // let b = prompt('请输入一个数字:')
21     // if(a > b){
22     //     document.write('a比b大')
23     // }else{
24     //     document.write('b比a大')
25     // }
26
27     // 多分支选择结构
28     // let score = prompt('请输入你的成绩:')
29     // if(score > 90){
30     //     document.write('优秀')
31     // }else if(score >= 80){
32     //     document.write('🐮')
33     // }else if(score >= 70){
34     //     document.write('👍')
35     // }else if(score >= 60){
36     //     document.write('🌺')
37     // }else{
38     //     document.write('继续努力')
39     // }
40
41     // 多分支结构（匹配）
42     // let num1 = prompt('请输入第一个要计算的数字:')
43     // let sy = prompt('请输入要计算的符号+、-、*、/:')
44     // let num2 = prompt('请输入第二个要计算的数字:')
45     // let result = 0
46     // switch(sy){
47     //     case '+':
48     //         result = Number(num1) + Number(num2)
49     //         break
50     //     case '-':
51     //         result = num1 - num2
52     //         break
53     //     case '*':
54     //         result = num1 * num2
55     //         break
56     //     case '/':
57     //         result = num1 / num2
58     //         break
59     //     default:
60     //         result = '输入有误，请重新输入!'
61     // }
62     // document.write(result)
63
64     // JavaScript中数据类型转换
65     // 隐式数据类型转换，再使用某些运算符的时候，会把不同的类型默认转成相同的类型来计算 -、
    *、/、>、<、==

```

```

66         // console.log(10 + '10') // 1010
67         // console.log(10 - '5') // 5
68         // console.log('10' * '2') // 20
69
70         // NaN (not a number) 非数字，在数据类型转换或者计算的时候，类型转换后无法呈现对应的数字
        状态，就返回NaN
71         // console.log(Number('10'))
72         // console.log(Number('10px'))
73
74         console.log(typeof NaN) // number
75     </script>
76 </body>
77 </html>

```

2、计算器升级

```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>Document</title>
7  </head>
8  <body>
9      <input type="text" class="txt1">
10     <select>
11         <option value="+">+</option>
12         <option value="-">-</option>
13         <option value="*">*</option>
14         <option value="/">/</option>
15     </select>
16     <input type="text" class="txt2">
17     <button class="calc">=</button>
18     <input type="text" class="result">
19
20
21     <button class="clear">clear</button>
22
23     <!-- 把外部的js文件引入到html文件 -->
24     <script src="../js/tools.js"></script>
25     <script>
26         // const txt1 = document.querySelector('.txt1')
27         // const txt2 = document.querySelector('.txt2')
28         // const btn = document.querySelector('.calc')
29         // const result = document.querySelector('.result')
30         // const clear = document.querySelector('.clear')
31
32         // const select = document.querySelector('select')
33
34         // js选择器简化
35         // 第二步：在进入函数体之前，进行形参赋值 ele = '.txt1'

```

```

36         // function $(ele){
37         //     // 第三步：在执行返回值之前，先执行 document.querySelector('.txt1')函数，进行
函数调用，并实参传递
38         //     // 第四步：在进入函数体之前，进行形参赋值
39         //     // 第五步：把获取到的元素以返回值的形式返回 <input type="text" class="txt1">
40         //     // 第六步：$函数通过return把结果返回给外界
41         //     return document.querySelector(ele) // return <input type="text"
class="txt1">
42         // }
43
44         // 第一步：$('.txt1') 调用$函数，并进行实参传递
45         // const txt1 = $('.txt1')
46         // const txt2 = $('.txt2')
47         // console.log(txt1, txt2)
48
49         // 获取元素
50         const txt1 = $('.txt1')
51         const txt2 = $('.txt2')
52         const btn = $('.calc')
53         const result = $('.result')
54         const select = $('select')
55         const clear = $('.clear')
56
57         btn.onclick = function(){
58             switch(select.value){
59                 case '+':
60                     result.value = Number(txt1.value) + Number(txt2.value)
61                     break
62                 case '-':
63                     result.value = txt1.value - txt2.value
64                     break
65                 case '*':
66                     result.value = txt1.value * txt2.value
67                     break
68                 case '/':
69                     result.value = txt1.value / txt2.value
70                     break
71             }
72         }
73
74         clear.onclick = function(){
75             txt1.value = ''
76             txt2.value = ''
77             result.value = ''
78         }
79     </script>
80 </body>
81 </html>

```

十一、选择器封装

```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>Document</title>
7  </head>
8  <body>
9      <input type="text" class="txt1">
10     <select>
11         <option value="+">+</option>
12         <option value="-">-</option>
13         <option value="*">*</option>
14         <option value="/">/</option>
15     </select>
16     <input type="text" class="txt2">
17     <button class="calc">=</button>
18     <input type="text" class="result">
19
20
21     <button class="clear">clear</button>
22
23     <!-- 把外部的js文件引入到html文件 -->
24     <script src="../js/tools.js"></script>
25     <script>
26         // const txt1 = document.querySelector('.txt1')
27         // const txt2 = document.querySelector('.txt2')
28         // const btn = document.querySelector('.calc')
29         // const result = document.querySelector('.result')
30         // const clear = document.querySelector('.clear')
31
32         // const select = document.querySelector('select')
33
34         // js选择器简化
35         // 第二步：在进入函数体之前，进行形参赋值 ele = '.txt1'
36         // function $(ele){
37         //     // 第三步：在执行返回值之前，先执行 document.querySelector('.txt1')函数，进行
函数调用，并实参传递
38         //     // 第四步：在进入函数体之前，进行形参赋值
39         //     // 第五步：把获取到的元素以返回值的形式返回 <input type="text" class="txt1">
40         //     // 第六步：$函数通过return把结果返回给外界
41         //     return document.querySelector(ele) // return <input type="text"
class="txt1">
42         // }
43
44         // 第一步：$('.txt1') 调用$函数，并进行实参传递
45         const txt1 = $('.txt1')
46         const txt2 = $('.txt2')
47         console.log(txt1, txt2)
48
49         // btn.onclick = function(){
50         //     console.log(select.value)

```

```

51         // }
52
53         // clear.onclick = function(){
54         //     txt1.value = ''
55         //     txt2.value = ''
56         //     result.value = ''
57         // }
58     </script>
59 </body>
60 </html>

```

十二、移动适配方案

- 1、pc端网站的布局使用的是 `px` 作为单位，是没有实现网页自适应，在移动端需要完全实现自适应
- 2、rem单位，是一个相对于根标记字体大小的单位

```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>Document</title>
7      <style>
8          html{
9              /* 表示1rem等于100px */
10             font-size: 100px;
11         }
12         .box{
13             width: 1rem;
14             height: 1rem;
15             background-color: hotpink;
16         }
17     </style>
18 </head>
19 <body>
20     <div class="box"></div>
21 </body>
22 </html>

```

- 3、使用js来动态计算rem的值

```

1  (function flexible (window, document) {
2      var docEl = document.documentElement
3      var dpr = window.devicePixelRatio || 1
4
5      // adjust body font size
6      function setBodyFontSize () {
7          if (document.body) {
8              document.body.style.fontSize = (12 * dpr) + 'px'
9          }

```

```
10     else {
11         document.addEventListener('DOMContentLoaded', setBodyFontSize)
12     }
13 }
14 setBodyFontSize();
15
16 // set 1rem = viewWidth / 10
17 function setRemUnit () {
18     var rem = docEl.clientWidth / 10
19     docEl.style.fontSize = rem + 'px'
20 }
21
22 setRemUnit()
23
24 // reset rem unit on page resize
25 window.addEventListener('resize', setRemUnit)
26 window.addEventListener('pageshow', function (e) {
27     if (e.persisted) {
28         setRemUnit()
29     }
30 })
31
32 // detect 0.5px supports
33 if (dpr >= 2) {
34     var fakeBody = document.createElement('body')
35     var testElement = document.createElement('div')
36     testElement.style.border = '.5px solid transparent'
37     fakeBody.appendChild(testElement)
38     docEl.appendChild(fakeBody)
39     if (testElement.offsetHeight === 1) {
40         docEl.classList.add('hairlines')
41     }
42     docEl.removeChild(fakeBody)
43 }
44 }(window, document))
```